VIT
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

REG.NO.:

SLOT:G1+TG1

**SCHOOL OF COMPUTER SCIENCE AND ENGINEERING**
**CONTINUOUS ASSESSMENT TEST - II**
**FALL SEMESTER 2025-2026**

---

**Programme Name & Branch** : **B.Tech CSE and Specialization**
**Course Code and Course Name** : **BCSE355L and Cloud Architecture Design**
**Faculty Name(s)** : **Common to All**
**Class Number(s)** : **Common to All**
**Date of Examination** : **11.10.2025**
**Exam Duration** : **90 minutes** **Maximum Marks: 50**

**General instruction(s):**

- Answer All Questions
- M - Max mark; CO – Course Outcome; BL – Blooms Taxonomy Level (1 – Remember, 2 – Understand, 3 – Apply, 4 – Analyse, 5 – Evaluate, 6 – Create)
- Course Outcomes:

  CO3: Optimize cost and performance by selecting appropriate AWS services and utilizing cost-effective resource management strategies
  CO4 Configure and manage advanced AWS networking features, storage solutions, database technologies, and compute resources.

| Q. No | Question | M | CO | BL |
|---|---|---|---|---|
| 1. | Two departments of a university maintain separate VPCs. Department A hosts the student portal, while Department B manages the exam system. Both applications require private communication without using the internet.<br>i. Explain how VPC Peering can be configured to enable connectivity between the two VPCs, highlighting the steps related to establishing the peering connection and configuring security group rules.<br>ii. Compare Site-to-Site VPN with VPC Peering in terms of security, latency, and scalability.<br>iii. Mention one limitation of VPC Peering that makes Site-to-Site VPN a preferable option.<br>iv. With the help of a neat diagram, represent the architecture of both VPC Peering and Site-to-Site VPN. | 10 | CO4 | 4 |
| 2. | An online learning platform stores student profiles, course content, and exam results in a relational database hosted on Amazon RDS. During normal operations, the system needs to ensure high availability so that the student portal remains functional even in the event of infrastructure failure. However, during exam periods, the platform experiences a sharp increase in read requests, as thousands of students simultaneously access progress reports and test results. As a cloud architect, explain how RDS Multi-AZ deployments and RDS Read Replicas can be combined to meet these requirements. Justify the role of each feature, and illustrate your answer with a neat diagram showing the primary database, standby instance across Availability Zones, and one or more read replicas serving high-volume read traffic. | 10 | CO4 | 3 |
| 3. | A gaming application needs to store player profiles, scores, and session data with millisecond latency, and the architects are considering Amazon DynamoDB for this purpose. Discuss why DynamoDB is more suitable than RDS for this use case, and describe how partition keys and sort keys help in organizing data | 10 | CO4 | 4 |

**SCHOOL OF COMPUTER SCIENCE AND ENGINEERING**
**CONTINUOUS ASSESSMENT TEST - II**
**FALL SEMESTER 2025-2026**

---

| | | | | |
|---|---|---|---|---|
| | efficiently within DynamoDB. Further, mention one trade-off of using DynamoDB compared to RDS. Finally, illustrate your answer with a neat diagram of DynamoDB showing items, attributes, and partitioning. | | | |
| 4. | In a cloud environment, organizations must ensure compliance by monitoring API activities and configuration changes. Explain how logging services can be used to track user actions and provide details on who made a change, when it occurred, and from where. Describe how configuration management services can record resource changes and enforce compliance policies, such as encryption requirements. Mention one benefit of integrating logging with configuration management for auditing purposes, and illustrate your answer with a neat diagram showing event logging, configuration tracking, and compliance enforcement. | 10 | CO3 | 3 |
| 5. | An ed-tech startup is preparing for rapid growth and aims to control costs, strengthen security, and improve fault tolerance without relying on additional cloud consultants. Explain how AWS Trusted Advisor can assist in identifying opportunities for cost optimization. Describe the ways in which Trusted Advisor provides security and performance checks. Finally, discuss the importance of regularly reviewing the Trusted Advisor dashboard and how it contributes to long-term cloud governance and operational efficiency. | 10 | CO3 | 2 |

**********

CAT-II Keys

**Q1)**

**i) VPC Peering Configuration Steps (4M)**

**Concept:**

VPC Peering enables private communication between two VPCs using AWS internal network without traversing the public Internet.

**Configuration Steps:**

1. **Create VPC Peering Connection:**
   - In the AWS Management Console → VPC Dashboard → *Peering Connections* → *Create Peering Connection*.
   - Choose *Requester VPC* (Dept A) and *Accepter VPC* (Dept B).
   - Specify account and region (can be same or different regions).
   - Click *Create Peering Connection* → *Accept Request* from Dept B.
2. **Update Route Tables:**
   - Add routes in each VPC's route table to direct traffic to the peer's CIDR block through the Peering Connection ID.
3. **Modify Security Groups:**
   - Allow inbound and outbound traffic from the peer VPC's CIDR range for required ports (e.g., HTTP/HTTPS, DB).
   - Example:
     - Dept A SG inbound rule → source: Dept B CIDR
     - Dept B SG inbound rule → source: Dept A CIDR

**Key Point:** Traffic between VPCs remains private — no internet gateway or NAT required.

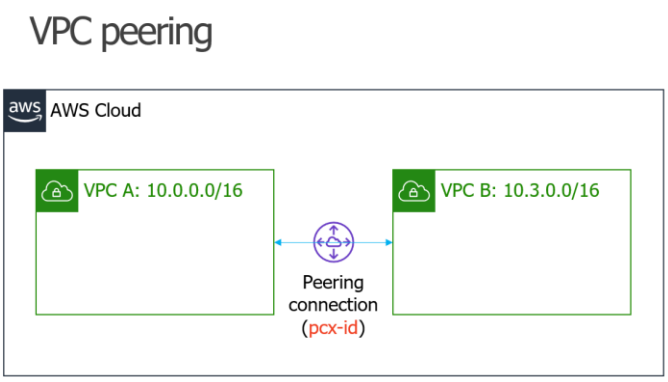
**ii) Comparison: Site-to-Site VPN vs VPC Peering(3M)**

| Feature | VPC Peering | Site-to-Site VPN |
|---|---|---|
| **Security** | Uses AWS internal network — inherently secure and private | Uses encrypted IPSec tunnel over the Internet |
| **Latency** | Lower latency due to direct AWS backbone routing | Higher latency due to Internet routing |
| **Scalability** | Point-to-point (not transitive); grows complex with many VPCs | More scalable — can connect multiple networks or on-premises to AWS |
| **Use Case** | VPC-to-VPC communication within or across regions | Hybrid or multi-site connectivity between AWS and on-prem |

**iii. Limitation of VPC Peering(1M)**

- No Transitive Routing:
  - Traffic cannot pass through a third VPC.
  - Example: VPC A ↔ VPC B ↔ VPC C → A cannot reach C through B.
  - Hence, for hub-and-spoke or large network topologies, Site-to-Site VPN or AWS Transit Gateway is preferable.

iv. Diagram(2M)



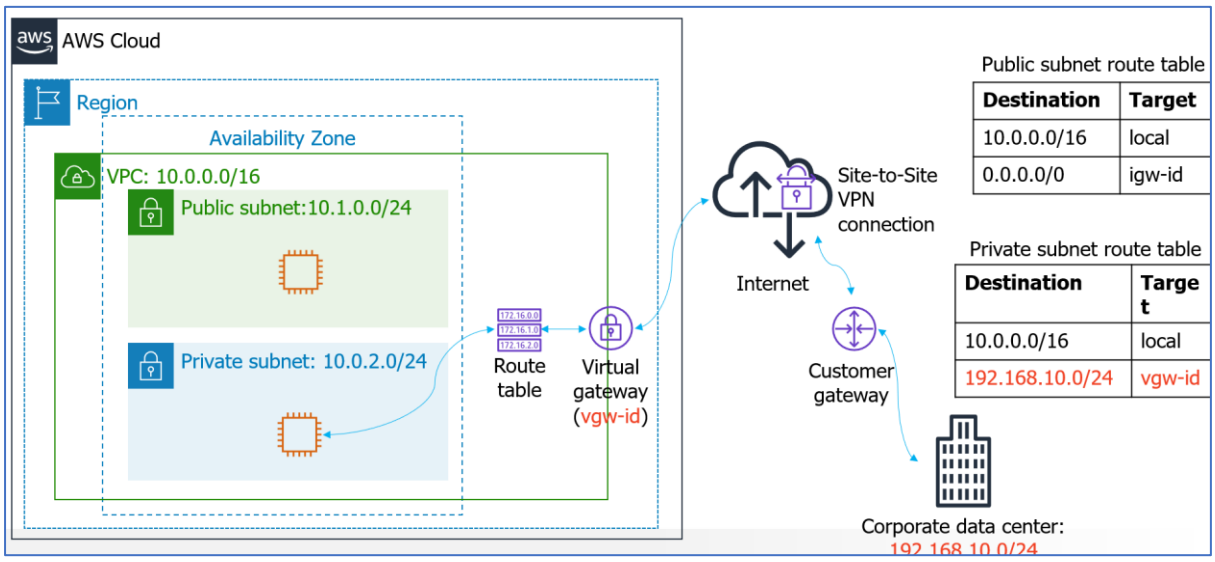

Route Table for VPC A

| Destination | Target |
|---|---|
| 10.0.0.0/16 | local |
| 10.3.0.0/16 | pcx-id |

Route Table for VPC B

| Destination | Target |
|---|---|
| 10.3.0.0/16 | local |
| 10.0.0.0/16 | pcx-id |

**Site to Site VPN**




---

Q2) **Scenario Understanding**

The platform must achieve:

1. **High Availability (HA)** – Database should automatically failover if an instance or Availability Zone (AZ) fails.
2. **Scalability for Read Traffic** – Handle spikes in read operations during exam periods.

Both goals can be achieved by **combining RDS Multi-AZ deployments** with **RDS Read Replicas**.

**i. Role of RDS Multi-AZ Deployment (High Availability) – 2m**

- **Purpose:** Ensures **automatic failover** and **data durability** for the primary database instance.
- **How it works:**
    - AWS RDS automatically provisions a **synchronous standby replica** in a **different Availability Zone** (AZ).
    - The standby instance continuously replicates data from the primary in **real time**.
    - In case of hardware failure, storage corruption, or AZ outage, **RDS automatically promotes the standby** to primary, minimizing downtime.
- **Key Characteristics:**
    - Synchronous replication → Ensures data consistency.
    - Not used for read queries (standby remains passive).
    - Automatic DNS failover managed by RDS.

**Benefit:** Keeps the **student portal always available**, even during infrastructure or instance failures.

**ii. Role of RDS Read Replicas (Scalability for Read Traffic)-2m**

- **Purpose:** Improves **read performance and scalability** during high-load situations (e.g., exams).
- **How it works:**
    - AWS creates **asynchronous copies** of the primary database (or its Multi-AZ setup).
    - These replicas can serve **read-only queries** (e.g., progress reports, test results).
    - Multiple replicas can be deployed **within the same region or across regions**.
- **Key Characteristics:**
    - Asynchronous replication → minimal lag (eventual consistency).
    - Each replica has its own endpoint for distributing read traffic.
    - Can be promoted to standalone DB in disaster scenarios.

**Benefit: Offloads read workload** from the primary, ensuring smooth performance during peak exam access.

**iii. Combined Architecture Explanation(2m)**

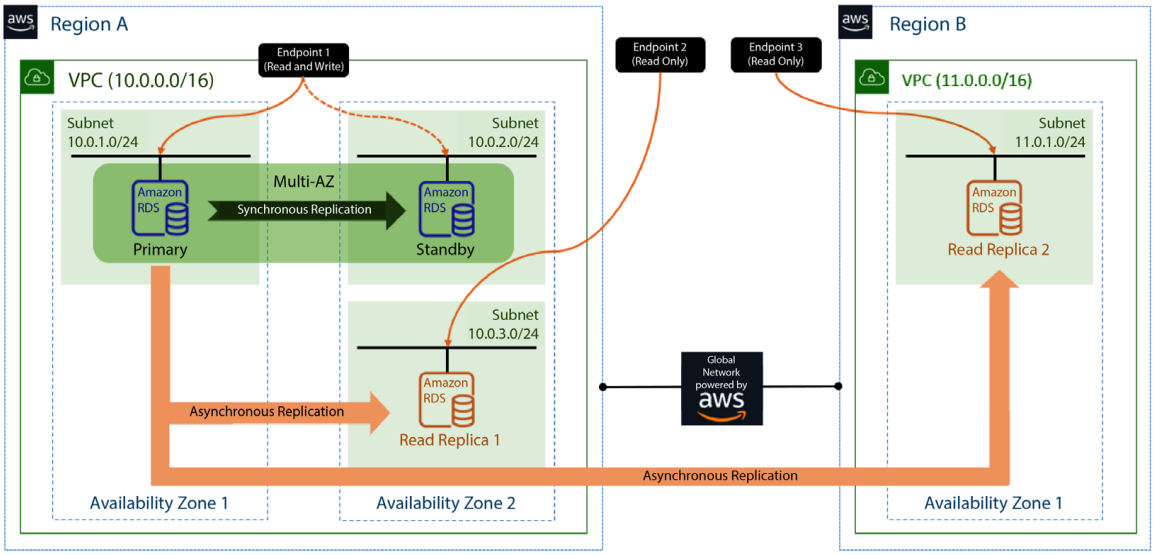
- **Primary DB**: Handles all write and transactional operations (student updates, course uploads).
- **Multi-AZ Standby**: Provides high availability and automatic failover.
- **Read Replicas**: Handle read-heavy operations during exam time to scale horizontally.

**Workflow Summary:**

1. Application writes → Primary DB.
2. Synchronous replication → Standby (for HA).
3. Asynchronous replication → Read Replicas (for load balancing).

4. During heavy read load → App routes queries to Read Replicas via a load balancer or read endpoint.
5. If primary fails → Standby promoted automatically; replicas continue replication from new primary.

iv) Diagram (2m)




v) Justification(2m)

| Feature | Purpose | Replication Type | Use Case |
|---|---|---|---|
| **Multi-AZ Deployment** | High Availability, Disaster Recovery | Synchronous | Ensures uptime during failures |
| **Read Replicas** | Performance, Scalability | Asynchronous | Serves large read workloads |

Q3)

**Scenario Understanding**

The gaming application requires millisecond latency, high scalability, and efficient handling of unstructured, high-volume data like player profiles, scores, and session logs. Hence, Amazon DynamoDB, a fully managed NoSQL key-value and document database, is more suitable than Amazon RDS, which is relational.

**Why DynamoDB is More Suitable than RDS?**

| Feature | Amazon DynamoDB | Amazon RDS |
|---|---|---|
| **Performance** | Provides **single-digit millisecond latency** for reads/writes, ideal for real-time gaming apps | Higher latency due to SQL query parsing and relational joins |
| **Scalability** | Horizontally scalable; automatically partitions data to handle millions of requests per second | Vertically scalable (limited by instance size) |
| **Data Model** | Schema-less key-value/document model suits dynamic data like player stats, session logs | Fixed schema; not ideal for frequently changing attributes |
| **Availability** | Built-in multi-region replication and on-demand capacity scaling | Requires manual configuration for replicas or read scaling |
| **Cost Efficiency** | Pay-per-request model, ideal for variable workloads like gaming sessions | Continuous instance cost even during low activity |

**Role of Partition Keys and Sort Keys in Data Organization**

**1. Partition Key (Primary Hash Key):**

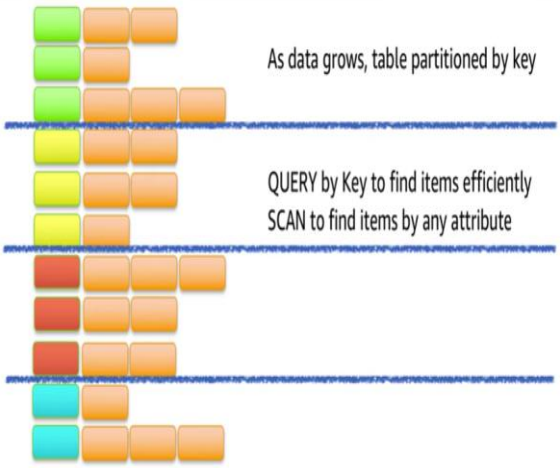
- Determines the **partition (physical storage location)** of each item.
- DynamoDB uses the partition key's hash value to distribute data evenly across multiple storage partitions for high performance.
- Example: PlayerID as the partition key ensures that all data related to one player is stored together.

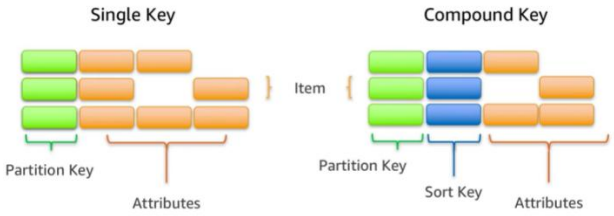
**2. Sort Key (Range Key):**

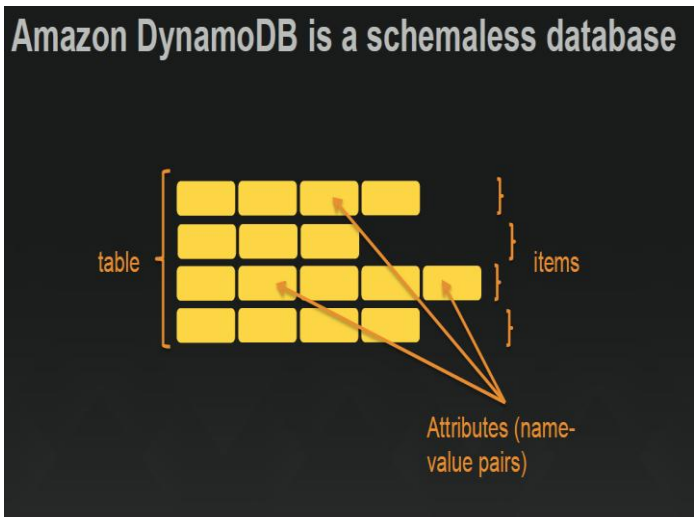
- Allows multiple items with the same partition key to be stored and sorted.
- Enables **querying and range-based retrievals** (e.g., latest scores, sessions).
- Example: SessionTimestamp as the sort key lets you fetch all sessions of a player in chronological order.

They form a **composite primary key** → (PlayerID, SessionTimestamp), supporting both **fast lookups** and **efficient range queries**.

**Diagram:**




**Summary:**

| Aspect | DynamoDB | RDS |
|---|---|---|
| **Data Model** | NoSQL (Key-Value/Document) | Relational (Tables, Schema) |
| **Performance** | Millisecond latency | Higher latency |
| **Scalability** | Automatic horizontal scaling | Limited vertical scaling |
| **Flexibility** | Schema-less | Fixed schema |
| **Trade-off** | Limited query features | Supports full SQL |

**Q4)**

In cloud environments, security, compliance, and governance require visibility into every user and system action.
Organizations achieve this through a combination of:

- Logging services → track who did what, when, and from where.
- Configuration management services → record what changed in the infrastructure and ensure policies are followed.

In AWS, these are represented by services such as AWS CloudTrail (for logging) and AWS Config (for configuration tracking and compliance).

**i. Role of Logging Services (Tracking User Actions)**
**Purpose:** To capture and store detailed records of every API call or console action performed in the cloud account.
**How it Works:**

- Every action made via AWS Management Console, CLI, or SDK is logged.
- Each log entry includes:
  - **Who**: The IAM user, role, or service that performed the action.
  - **When**: The exact timestamp of the event.
  - **What**: The specific API call or resource change.
  - **Where**: The source IP address or AWS Region.

**Example:**
If a user deletes an S3 bucket, CloudTrail records:
DeleteBucket event → User: "AdminUser", Time: "2025-10-11 09:30:15 UTC", Source IP: "10.1.2.3".
**Benefit:** Provides **accountability and traceability** for security and compliance audits.

**ii. Role of Configuration Management Services (Tracking and Enforcing Compliance)**
**Purpose:** To continuously record and evaluate the state of cloud resources.
These services ensure that the environment aligns with internal or regulatory compliance standards (e.g., HIPAA, PCI-DSS).
**How it Works:**

- Captures and stores **resource configuration snapshots** (e.g., S3 buckets, EC2 instances, IAM roles).
- **Tracks changes** over time — what configuration changed, and when.
- Allows defining **Config Rules / Policies** such as:
  - "All S3 buckets must have server-side encryption enabled."
  - "No security group should allow public SSH access (port 22)."
- If a violation occurs, the system triggers **alerts or automated remediation**.

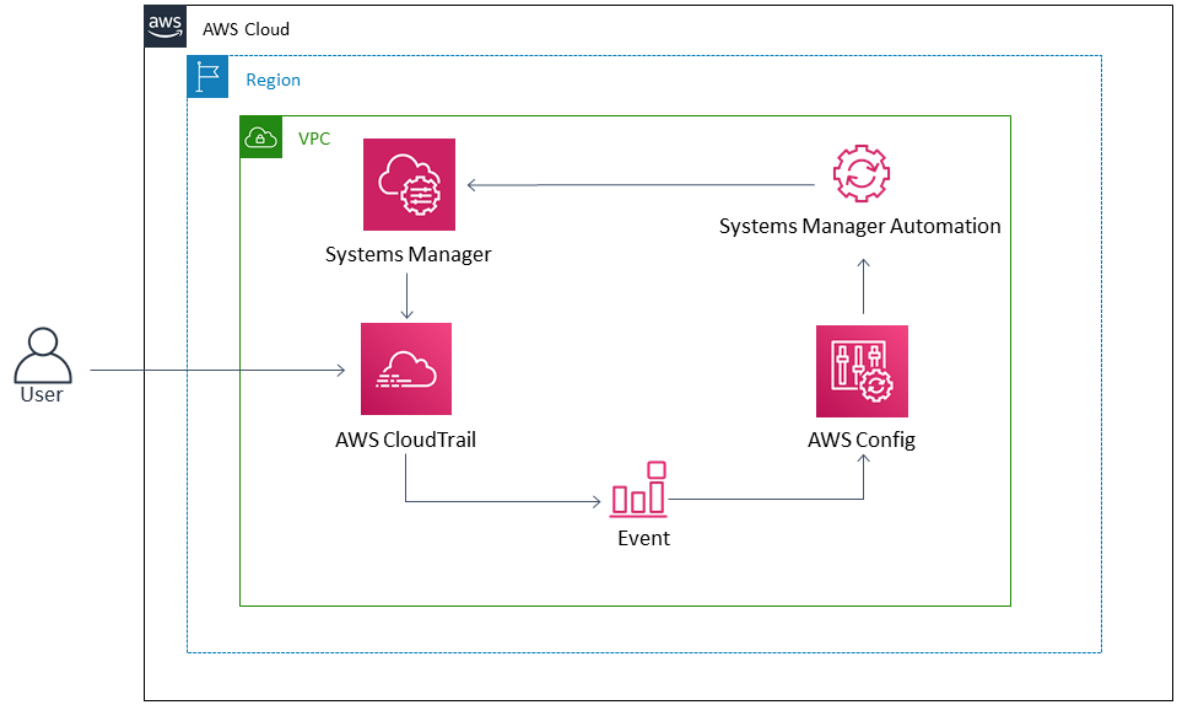
**Service Example:** *AWS Config*, *Azure Policy*, *Google Cloud Config Controller*.

**iii. Benefit of Integrating Logging with Configuration Management**
**Integrated Advantage:**
Combining logging and configuration management gives a complete compliance audit trail.

**iii. Benefit of Integrating Logging with Configuration Management**
Integrated Advantage:
Combining logging and configuration management gives a complete compliance audit trail.

| Aspect | Logging Only | Logging + Config Integration |
|---|---|---|
| **Visibility** | Shows *who* performed the change | Also shows *what changed* in resource configuration |
| **Compliance Enforcement** | Passive tracking | Active detection + auto-remediation |
| **Auditing Capability** | Event-based | End-to-end traceability from action → configuration → compliance status |

Diagram:




| Feature | Logging Service (e.g., CloudTrail) | Config Management (e.g., AWS Config) |
|---|---|---|
| **Primary Function** | Record API calls and actions | Record and evaluate resource states |
| **Tracks** | Who, When, What, Where | What changed, and compliance status |
| **Use Case** | Auditing user activity | Policy enforcement and governance |
| **Integration Benefit** | Full audit trail linking user actions to resource compliance | |

Q5)

The ed-tech startup is scaling rapidly on AWS and must balance **cost control**, **security**, and **fault tolerance** while maintaining efficiency **without hiring external consultants**. **AWS Trusted Advisor** acts as a **virtual cloud expert**, automatically analyzing AWS environments and providing actionable recommendations across **five key pillars**:

- Cost Optimization
- Performance
- Security
- Fault Tolerance
- Service Limits

**i. Role of AWS Trusted Advisor in Cost Optimization**

**Purpose:** To help the startup identify areas where AWS resources can be used more efficiently and at lower cost.

**How it Works:**
Trusted Advisor continuously scans the AWS environment and suggests **cost-saving opportunities**, such as:

1. **Idle or Underutilized Resources:**
    - Detects EC2 instances running at low utilization.
    - Recommends resizing, stopping, or terminating idle instances.
2. **Unattached Resources:**
    - Finds unused Elastic IPs, unattached EBS volumes, or idle load balancers.
    - Suggests deletion to avoid unnecessary billing.
3. **Right-Sizing and Purchasing Plans:**
    - Advises on purchasing **Reserved Instances** or **Savings Plans** based on usage trends.
4. **S3 and Data Transfer Optimization:**
    - Identifies unused or duplicate storage classes; recommends moving infrequently accessed data to **S3 Glacier or S3 IA**.

**Benefit:** Helps the startup **reduce operational expenses** while maintaining performance and scalability.

**ii. Security and Performance Checks by Trusted Advisor**
**1. Security Checks:**
Trusted Advisor helps maintain a secure AWS environment by identifying **potential vulnerabilities** and **misconfigurations** such as:

- **IAM Best Practices:** Detects accounts with no MFA or overly permissive IAM policies.
- **S3 Bucket Permissions:** Alerts if public access is enabled unintentionally.
- **Security Groups:** Flags unrestricted ports (e.g., 0.0.0.0/0 access).
- **Root Account Usage:** Warns when the root account is used instead of IAM roles.
- **SSL/TLS Certificates:** Tracks expiring or expired certificates.

**Outcome:** Reduces risk of breaches and ensures compliance with data protection policies.
**2. Performance Checks:**
Ensures applications and infrastructure deliver **optimal speed and reliability** by analyzing:

- **EC2 Instance Types and Utilization:** Suggests upgrades/downgrades for performance efficiency.
- **Auto Scaling Group Health:** Recommends configurations for dynamic scaling.
- **RDS Optimization:** Identifies underperforming database instances.
- **Service Limits:** Warns when resource usage nears AWS account limits (e.g., EC2 instances per region).

**Outcome:** Improves application responsiveness and fault tolerance during peak usage

**iii. Importance of Regularly Reviewing the Trusted Advisor Dashboard**
**1. Continuous Improvement and Governance:**
Regularly reviewing the dashboard ensures that the organization stays aligned with **AWS Well-Architected Framework** principles. It enables proactive governance and ongoing compliance monitoring.
**2. Cost and Resource Optimization Over Time:**
Periodic reviews help identify new unused resources or inefficiencies as workloads evolve, preventing cost creep.

**3. Security Hygiene and Incident Prevention:**
Regular checks prevent drift from security best practices — ensuring policies such as MFA enforcement, encryption, and least privilege access are continuously upheld.
**4. Operational Efficiency:**
Teams can automate fixes based on Trusted Advisor alerts using **AWS Lambda or Systems Manager Automation**, reducing manual intervention.
**Overall Impact:**
Regular Trusted Advisor reviews create a **self-auditing ecosystem**, ensuring **long-term efficiency**, **financial control**, and **resilient cloud operations** — essential for a rapidly growing startup.

| Aspect | Description | Example / Outcome |
|---|---|---|
| **Cost Optimization** | Identifies unused or underutilized resources | Stop idle EC2 instances |
| **Security Checks** | Flags risky IAM, S3, or network settings | Enforce MFA, restrict ports |
| **Performance Checks** | Monitors instance and database performance | Resize RDS or EC2 instances |
| **Dashboard Review** | Enables continuous improvement | Reduces cost, enhances reliability |